Real-time detection of fraudulent credit-card transactions using a temporal convolutional autoencoder hybrid model



Candidate Number: [redacted]

Mathematics and Physical Sciences

University of Sussex

A thesis submitted for the degree of

MSc Human and Social Data Science

August 2026

Contents

1 Introduction **1**

1.1 Background and motivation 1
1.2 Problem statement 3
1.3 Research Questions 3
1.4 Scope and Limitations 4
1.5 Thesis Structure 4

2 Literature Review **6**

2.1 Credit Card Fraud Detection as a Machine Learning Problem 6
2.2 Class Imbalance and Cost-Sensitive Learning 7
2.3 Autoencoders for Anomaly and Fraud Detection 7
2.4 Sequence Modelling: Recurrent Networks, WaveNet, and Temporal Convolutional Networks 8
2.5 Gradient-Boosted Decision Trees and XGBoost 9
2.6 Hybrid Deep Learning and Gradient Boosting Architectures 10
2.7 Evaluation Methodology for Imbalanced, Time-Ordered Data 10

3 Dataset **12**

3.1 Dataset Overview 12
3.2 Temporal Structure 12
3.3 Feature Space and Missingness 13
3.4 Class Balance Across Splits 13
3.5 Implications for Modelling 14

4 Methodology **15**

4.1 System Overview 15
4.2 Data Loading and Time-Based Splitting 16
4.3 Feature Engineering Pipeline 16
4.3.1 V-column reduction 17

4.3.2 Time and account features . . . 17
4.3.3 Derived features . . . 17
4.3.4 Missingness indicators . . . 18
4.3.5 Frequency encoding . . . 18
4.3.6 Categorical and numeric encoding . . . 18
4.3.7 Causal per-account aggregates . . . 18
4.4 Sliding-Window Sequence Construction . . . 19
4.5 Temporal Convolutional Autoencoder Architecture . . . 20
4.5.1 Motivation for architectural choices . . . 20
4.5.2 Temporal block . . . 20
4.5.3 Stack and dilation schedule . . . 20
4.5.4 Padding-aware masking . . . 21
4.5.5 Encoder and latent bottleneck . . . 21
4.5.6 Decoder and reconstruction . . . 21
4.5.7 Reconstruction loss and masking . . . 21
4.6 Feature Fusion: From Reconstruction to a Classifier-Ready Representation . . . 22
4.7 Feature Selection: RFECV . . . 22
4.8 XGBoost Classifier . . . 23
4.8.1 Custom focal-loss objective . . . 23
4.8.2 Early stopping metric . . . 24
4.9 Account-Level Fraud Cascade . . . 24
4.10 Probability Calibration and Threshold Selection . . . 24
4.10.1 Isotonic calibration . . . 25
4.10.2 F-beta-optimal threshold . . . 25
4.10.3 Bootstrap threshold stabilisation . . . 25
4.11 Held-Out Test Evaluation and Uncertainty Quantification . . . 25
4.12 Inference Artefacts and Real-Time Deployment Considerations . . . 26

5 Experimental Setup and Implementation 27
5.1 Software and Hardware . . . 27
5.2 Reproducibility . . . 27
5.3 Hyperparameter Summary . . . 27
5.4 Autoencoder Checkpointing and Training Behaviour . . . 28
5.5 Evaluation Protocol Summary . . . 29

6 Results **30**
6.1 Dataset and Feature Engineering Summary 30
6.2 Autoencoder Training 30
6.3 Feature Selection 31
6.4 XGBoost Classifier Training 31
6.5 Calibration and Threshold Selection 32
6.6 Held-Out Test Performance 32
6.7 Uncertainty Quantification on the Test Metric 33
6.8 Summary of Findings 33

7 Discussion **35**
7.1 Interpreting the Ranking Performance 35
7.2 The Precision-Recall Trade-off at the Selected Operating Point 35
7.3 The Effect of the Fraud Cascade Rule 36
7.4 Relationship to the Research Questions 37
7.5 Limitations 38

8 Conclusion and Future Work **40**
8.1 Summary of the Thesis 40
8.2 Answering the Research Questions 41
8.3 Directions for Future Work 41
8.3.1 Ablation of the autoencoder contribution 41
8.3.2 Cost-sensitive, institution-specific threshold selection 41
8.3.3 Graph-based and network-level features 42
8.3.4 True streaming evaluation 42
8.3.5 Repeated-seed variance estimation 43
8.4 Closing Remarks 43

Bibliography **44**

Chapter 1: Introduction

1.1 Background and motivation

Credit card fraud represents a persistent and costly challenge facing the global financial services industry. Card-not-present (CNP) fraud is the most common form of credit card fraud - accounting for approximately 79% of all credit-card fraud in the UK as of 2021 [1]. This accounts for £412.5 million of fraudulent transactions that were detected and reversed by UK banks, and has increased from 63% of all transactions since 2012 [1]. This increase likely results from the shift towards digital and contactless payments, with a further increase in fraud, particularly in cyber-enabled fraud such as credit-card fraud, occurring as a result of the Covid-19 pandemic and ensuing lockdown [2]. Beyond direct losses from fraudulent transactions, credit card fraud erodes trust, increases banking operational costs, and imposes new compliance burdens on banks and other payment providers that arise relating to new regulatory frameworks. The Payment Systems Regulator has introduced guidance applicable to banks and other payment systems providers requiring them to implement both a robust fraud detection system, such as the system created in this thesis, and a customer awareness and education program within their service [3]. These regulations also apply to digital payment platforms that do not hold consumers money and may only provide online payment services - such as fintech firms or some transaction processing companies.

These compliance regulations have led to the development of a variety of digital detection systems. Issuing banks and other payment providers must decide, often on a very large scale, whether an individual transaction should be approved, declined or sent for additional verification by the customer. Failing at this decision in either direction comes at a cost - declining legitimate transactions can be an inconvenience to the customer, while approving fraudulent transactions produces financial loss to either the bank or the merchant if caught by the customer.

The nature of credit-card fraud makes its detection, and therefore this decision, particularly difficult. Fraudulent transactions are relatively rare when compared to legitimate ones - they typically make up significantly under 1% of a total transaction volume within a dataset. For example, in the well-known Credit-Card Fraud dataset provided by ULB, which is the dataset utilized by this research, fraudulent transactions make up 0.172% of the data. This creates a severe class imbalance issue when considering an analytically driven detection model, which would typically need to be mitigated using one of a variety of strategies that will be later addressed. Additionally, fraud patterns are non-stationary: fraud typically adapts with its detection model, particularly as technology improves. This evolving nature of fraudulent transactions amounts from a variety of characteristics within the data that ultimately filter down to one major feature - human interference [4]. Human behaviour, both on the side of the fraudster and the victim's response, can change how fraudulent behaviour evolves. This interaction is also likely to be different across different types of fraud, and causes fraud characteristics to change rapidly. This ultimately leads to static, rule-based detection systems, such as decision trees or random forests, struggling to keep up with real-time detection. As a result of this, these models often generate high false-positive rates, which can be an inconvenience for customers in a real-world setting, or they may fail to catch new forms of fraud that have radically different characteristics.

This research is motivated by the observation that two families of models are historically applied to this problem separately. The first model is unsupervised and semi-supervised anomaly detection using autoencoders. These models are trained on normal behaviour and flag deviations from the learned behaviour using reconstruction error. This model therefore does not require a representative sample of fraud during the training window, and is highly interpretable in a way other deep learning models usually are not. However, reconstruction error is a one-dimensional summary of a very rich internal representation and its direct use as a classification score generally underperforms discriminative classifiers that have been shown labelled fraud examples. The second modelling family is supervised, discriminative modelling on hand-engineered tabular features, where gradient-boosted decision trees are used most often. These models' major weakness is having no native mechanism for capturing sequential, account-level features unless they are hand-engineered - which is a time-intensive process.

The hybrid architecture developed in this thesis is an attempt to combine the complementary strengths of these two models - a temporal convolutional autoencoder

(TCN-AE) is used as a learned, sequence-aware feature extractor, and the latent state and reconstruction diagnostics are fused with the original tabular features and fed into an XGBoost classifier trained on the fraud label. A TCN is used instead of more traditional recurrent architectures such as Long-Short Term Memory Networks (LSTM) because dilated causal convolutions provide a large receptive field with parallelisable training and stable gradients over long sequences, which are attractive properties for real-time deployment.

1.2 Problem statement

Given a stream of credit-card transactions, associated with a bespoke account identifier, timestamp, amount, and a variety of anonymised and semi-anonymised features, the task is to predict the probability that each transaction is fraudulent and to convert that probability into a binary fraud decision at a threshold chosen to reflect a trade-off between missed fraud and false alarms. This decision is made using only information available strictly before or at the time of the individual transactions, and the system is evaluated on a held-out set of transactions that occurred following the transactions used for training and calibration.

1.3 Research Questions

This thesis addresses four research questions:

1. Can a temporal convolutional autoencoder (TCN-AE), trained on only non-fraudulent transaction sequences, learn a latent representation and reconstruction-error signature that is discriminative of fraud when combined with a downstream classifier, beyond what the raw tabular features alone provide?

2. Does fusing autoencoder-derived features (latent embedding, terminal-step reconstruction error, window-level error statistics) with a gradient-boosted classifier trained under a custom focal-loss objective outperform a purely tabular baseline?

3. What is the effect of accounting for within-account correlation - via a causal "fraud cascade" rule that propagates a fraud flag forward once an account has been flagged - on the honesty and stability of the reported evaluation metrics?

4. How should a decision threshold be selected and stabilised, under extreme class imbalance and a limited number of positive examples in the calibration set, so that reported performance is not an artefact of a single lucky (or unlucky) validation split?

1.4 Scope and Limitations

The scope of this thesis is bounded in several respects. Firstly, it is important to note that the results of this research are not directly comparable to any models produced as part of the initial competition conducted using this data. This is because this research only uses the training transaction and identity tables, rather than the official test set, as the class labels for the test set were not publicly released and no label-based evaluation would be possible. The train/validation/test split of this data is described in Chapter 3.

Secondly, the model training and testing is conducted entirely offline, on historical data. The architecture is designed with real-time deployment as a key concern - bounded latency inference, streaming-compatible feature computation, and a fixed-length sliding window - however, the low-latency serving system is not deployed or load-tested.

Third, the pseudo-account identifier used in this work is a heuristic proxy for a true account identifier, which is not present in the original dataset as an anonymisation protection. This means that some genuine accounts will be split across multiple UIDs for reasons such as incorrect D1 account-age imputation, and some UIDs will conflate distinct accounts that happen to share card, address, and opening-day values. This is an unavoidable limitation of working with an anonymised public dataset and is further considered in Chapter 5. Fourth, the anonymised "V" columns, "C" columns, and "id_" columns do not have any semantic explanations, and therefore any feature engineering decisions are statistical rather than domain-informed.

1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 reviews the relevant literature on credit card fraud detection, class-imbalanced learning, autoencoder-based anomaly detection, temporal convolutional networks, and gradient-boosted decision trees, situating the present work relative to prior hybrid architectures. Chapter 3 describes the dataset used in this study, including its structure, class balance, and the

specific challenges of its anonymised feature space. Chapter 4 presents the methodology in detail - feature engineering pipeline, sliding window dataset construction, TCN autoencoder architecture, feature-fusion and feature-selection procedure, XGBoost training objective, the fraud cascade rule, and the calibration and thresholding procedure. Chapter 5 describes the experimental setup, including hyperparameters, training procedure, and implementation details. Chapter 6 presents the empirical results obtained from a full training run of the system, including validation and held-out test performance. Chapter 7 discusses these results in relation to the research questions and the wider literature, and Chapter 8 concludes with a summary of findings, limitations, and directions for future work.

Chapter 2: Literature Review

2.1 Credit Card Fraud Detection as a Machine Learning Problem

Automated credit card fraud detection has been studied since at least the 1990s, and the field has tracked the broader evolution of machine learning methodology: from rule-based expert systems, through classical statistical classifiers (logistic regression, naive Bayes), to ensemble tree methods, and more recently to deep learning. Across this evolution, three properties of the problem have remained constant and have shaped almost every methodological choice made in the literature. First, positive examples (fraud) are rare, typically well under 1% of transactions in production and around 0.1–5% in the public benchmark datasets most commonly used in academic work. Second, the cost of a false negative and the cost of a false positive are not symmetric, and are themselves not fixed but depend on transaction amount, customer relationship value, and operational capacity for manual review. Third, fraud is adversarial and non-stationary: fraud patterns change over time as fraudsters adapt to deployed defences, which makes any static offline evaluation an approximation of a moving target and motivates the use of time-respecting evaluation protocols rather than random cross-validation. A substantial body of work, associated in particular with Dal Pozzolo and colleagues [5], has focused specifically on the methodological pitfalls of naive evaluation under class imbalance, showing that metrics such as accuracy are essentially uninformative in this regime and that even AUC-ROC can be misleadingly optimistic when the positive class is very rare, because a large area under the ROC curve can coexist with poor precision at any threshold an institution could actually operate. This has led to a broad consensus in the more recent literature that precision-recall based metrics (PR-AUC, F1, F-beta with beta greater than one to reflect the higher cost typically assigned to missed fraud) are more appropriate primary metrics for this task, a position adopted throughout the present thesis.

2.2 Class Imbalance and Cost-Sensitive Learning

Two broad families of techniques have been proposed to address class imbalance: data-level methods, which resample the training distribution (random undersampling of the majority class, random or synthetic oversampling of the minority class such as SMOTE, or hybrid combinations), and algorithm-level methods, which modify the learning objective itself to weight errors on the minority class more heavily, without altering the data distribution the model observes. He and Garcia's [6] widely cited survey of learning from imbalanced data catalogues both families and highlights a persistent trade-off: oversampling techniques can improve recall but risk overfitting to synthetic or duplicated minority examples, while undersampling techniques discard majority-class information and can hurt the model's ability to characterise the boundary of the normal class precisely. Algorithm-level cost-sensitive learning has become particularly attractive in the era of gradient-boosted trees and neural networks because it can be implemented directly as a modification to the loss function's gradient, without altering the underlying data. The present work adopts this approach, using a combination of instance re-weighting (via inverse class frequency) and a focal-loss-style objective, discussed further below, applied to the XGBoost head. Focal loss was originally proposed by Lin and colleagues [7] in the context of dense object detection, where the overwhelming majority of candidate regions are easy negatives; the central idea is to down-weight the contribution of well-classified examples (of either class) via a modulating factor and up-weight the contribution of hard, misclassified examples, which has an effect closely analogous to what is needed in fraud detection, where the overwhelming majority of legitimate transactions are "easy negatives" that a well-trained model should classify with very high confidence and low gradient contribution, leaving training capacity for the comparatively rare, harder-to-separate fraud cases.

2.3 Autoencoders for Anomaly and Fraud Detection

Autoencoders — neural networks trained to reconstruct their own input through a lower-dimensional (or otherwise constrained) bottleneck — have been used for anomaly detection since well before the current deep learning era, on the logic that a model trained exclusively, or predominantly, on normal data will reconstruct normal inputs with low error and will generalise poorly to inputs drawn from a dif-

ferent distribution, producing a higher reconstruction error that can be used as an anomaly score. Sakurada and Yairi's early work on autoencoders for anomaly detection via nonlinear dimensionality reduction [8], and the subsequent large literature it spawned, established reconstruction error thresholding as a practical, unsupervised anomaly detection technique applicable to tabular and sequential data alike. In the specific context of credit card and financial fraud, autoencoders have most commonly been applied in one of two ways: (i) as a standalone anomaly detector, thresholding directly on reconstruction error, typically in a fully unsupervised setting where no fraud labels are available at training time; or (ii) as a representation-learning or feature-extraction front end, whose bottleneck (latent) representation, reconstruction error, or both, are fed into a downstream supervised classifier that does have access to labels. The present work follows the second pattern. The rationale for preferring representation extraction over direct reconstruction-error thresholding is well established in the anomaly detection literature: reconstruction error alone discards most of the information contained in the network's internal state, is a single scalar (or, at best, per-feature) summary that cannot represent complex, non-monotonic relationships between reconstruction behaviour and the true label, and, particularly for high-dimensional tabular inputs with many nearly-irrelevant or noisy columns, can be dominated by reconstruction difficulty on features that have nothing to do with fraud. Using the latent embedding directly as a learned feature space, and supplementing it with several complementary reconstruction-error statistics (rather than a single aggregate), allows a downstream discriminative model to learn a much richer, non-linear decision boundary over the autoencoder's internal representation, while retaining the benefit that the representation itself was learned in a way that is anchored to what "normal" behaviour looks like.

2.4 Sequence Modelling: Recurrent Networks, WaveNet, and Temporal Convolutional Networks

Transaction streams are naturally sequential — an account's spending behaviour at time t is informative context for interpreting a transaction at time t+1 — and a substantial literature has explored sequence models for fraud and related time-series anomaly detection tasks. Recurrent neural networks, and gated variants such as LSTM and GRU, were the default architecture for sequence modelling for much of the 2010s, but they are inherently sequential at training time (each time step depends on the previous hidden state), which limits parallelisation, and they are known

to suffer from optimisation difficulties over long sequences despite gating mechanisms designed to mitigate vanishing gradients. An alternative line of work, beginning with WaveNet's use of dilated causal convolutions for raw audio generation, showed that stacked convolutional layers with exponentially increasing dilation rates can achieve very large effective receptive fields while remaining fully parallelisable during training, since all time steps in a layer can be computed simultaneously rather than sequentially. Bai, Kolter, and Koltun's [9] empirical evaluation of "generic" temporal convolutional networks (TCNs) — combining dilated causal convolutions, residual connections, and weight normalisation — demonstrated that this architecture is a strong, and often superior, alternative to recurrent networks across a wide range of sequence modelling benchmarks, while offering more stable gradients and lower memory usage during training, motivating the adoption of TCNs as a general-purpose sequence architecture well beyond audio generation. The causal convolution property is particularly attractive for a real-time fraud detection setting, since it guarantees, by construction, that the representation computed at a given position in the sequence depends only on that position and earlier ones, which is exactly the "no future leakage" property required at inference time.

2.5 Gradient-Boosted Decision Trees and XGBoost

Gradient-boosted decision tree ensembles remain, as of this writing, the most widely used family of models for supervised learning on heterogeneous tabular data, routinely outperforming deep learning architectures on tabular benchmarks that lack strong spatial or sequential structure, and dominating industry deployments for exactly this kind of structured, mixed-type financial data. XGBoost, introduced by Chen and Guestrin, popularised a particularly efficient, regularised, and scalable implementation of gradient boosting, with native support for missing values, a histogram-based split-finding algorithm, and — importantly for the present work — a fully customisable training objective, expressed as a user-supplied function returning the gradient and Hessian of the loss with respect to the model's raw predictions. This customisability is what allows a focal-loss-style objective, which is not a standard built-in XGBoost loss, to be implemented directly as described in Chapter 4.

2.6 Hybrid Deep Learning and Gradient Boosting Architectures

A growing body of applied work has explored hybrid architectures that use a neural network as a learned feature extractor or representation-learning front end, with a gradient-boosted tree model as the final discriminative classifier, on the premise that neural representation learning and tree-based decision boundaries are complementary: the neural component can capture non-linear interactions and sequential or spatial structure that hand-engineered features miss, while the tree-based component is often more sample-efficient, more robust to irrelevant or noisy input dimensions, and more interpretable (via feature importance and SHAP-style attribution) than a fully end-to-end neural classifier, particularly on datasets of the size typically available for fraud detection, where a purely end-to-end deep network can be prone to overfitting the rare positive class. Randhawa and colleagues' study[10] combining multiple classical classifiers with AdaBoost-style majority voting for credit card fraud, and Fiore and colleagues' use of generative adversarial networks[11] to synthesise additional minority-class training examples for a downstream classifier, are representative examples of this general "hybridisation" strategy applied specifically to card fraud, albeit with different neural components than the temporal convolutional autoencoder used here. The architecture developed in this thesis extends this hybrid tradition by using a self-supervised (reconstruction-trained) sequence model, rather than a supervised or generative one, as the feature-extraction front end, which allows the representation-learning stage to use the very large pool of legitimate transactions without requiring balanced or synthetic fraud examples at that stage.

2.7 Evaluation Methodology for Imbalanced, Time-Ordered Data

Finally, a smaller but important strand of the literature addresses evaluation methodology specifically. Beyond the choice of metric discussed in Section 2.1, two further methodological issues are particularly relevant to this thesis. First, random k-fold cross-validation is inappropriate for time-ordered transaction data, since it can place a transaction in the training set while an earlier transaction on the same account, or an earlier instance of the same fraud pattern, falls in a validation fold evaluated as if it were "future" data relative to training — a form of temporal leakage that inflates reported performance relative to what a genuinely real-time deployed model would

achieve. The time-based, non-overlapping train/validation/test split used throughout this thesis is designed specifically to avoid this failure mode. Second, several authors have noted that when fraud is confirmed at the account or card level rather than purely at the transaction level, a single compromised account can generate a burst of correlated fraudulent transactions; failing to account for this correlation when computing transaction-level metrics can make a model appear to have higher recall than it would achieve at the point of the first fraudulent transaction on a newly compromised account, which is normally the operationally critical detection point. The fraud cascade mechanism described in Chapter 4 is this thesis's response to that concern.

Chapter 3: Dataset

3.1 Dataset Overview

The experiments in this thesis are conducted on the IEEE-CIS Fraud Detection dataset, which is a large, real-world, anonymised transaction dataset released for the IEEE Computational Intelligence Society fraud detection competition. The dataset was jointly provided by the Vesta Corporation. It consists of two linked tables: a transaction table, which contains one row per transaction with an identifier (`TransactionID`), a relative timestamp (`TransactionDT`, measured in seconds), a transaction amount (`TransactionAmt`), a fraud label (`isFraud`), a set of card-related fields (`card1` – `card6`), address fields (`addr1`, `addr2`), email domain fields, a large block of "D" (time-delta) features, "C" (counting) features, "M" (match) features, and 339 anonymised "V" features; and an identity table, containing device and connection metadata (browser, OS, device type, and further anonymised `id_01` – `id_38` fields) for a subset of transactions. The identity table is left-joined onto the transaction table on `TransactionID`.

After loading and joining, the working dataset contains 590,540 transactions, of which 20,663 are confirmed to be fraudulent (fraud base-rate: 3.499%). This rate is significantly higher than other fraud rates observed in live production payment streams, but remains a strongly imbalanced classification problem and is representative of the class of fraud detection problems addressed by this research.

3.2 Temporal Structure

The `TransactionDT` field is a relative timestamp, in seconds, measured from a reference point (the first transaction in the dataset) rather than an absolute calendar date. This feature is used to establish strict chronological ordering of the data. Following time-sorting, an hour of day feature and day-of-week feature are derived using the following formulae:

$\texttt{Transaction_hour} = \lfloor \texttt{Time} \div 3600 \rfloor \bmod 24 \texttt{Transaction_day} = \lfloor \texttt{Time} \div 86400 \rfloor \bmod 7$

These features allow the model to learn any diurnal or weekly patterns present in the data, even without an absolute calendar date.

3.3 Feature Space and Missingness

The raw feature space is unusually wide and sparse for a tabular fraud dataset. This is particularly true of the 339 anonymised V columns, which dominate the feature space and introduce redundancy and missingness - many of the columns have correlated missingness, and those columns within each missingness group typically have very high correlation with one another. The feature engineering pipeline described in Chapter 4 includes the dimensionality reduction methodology for these columns: they are grouped by their exact pattern of missingness, and within these groups, columns with correlation above 0.90 are collapsed to a single representative column. This methodology reduced the 339 raw V columns to 228 kept columns, of which one was dropped for zero variance and 110 were dropped as redundant across 14 distinct missingness groups.

Beyond the V block, 181 of the 334 non-protected (i.e. not identifying or labelling) columns remaining in the dataset following initial feature engineering have at least 30% of their values missing from the training split. Rather than removing these columns entirely and losing information from the dataset, a binary missingness indicator is added - on the assumption that whether or not a value is missing can be informative. Following feature engineering, the feature matrix used to train the TCN-AE contains 1,519 columns.

3.4 Class Balance Across Splits

The dataset is split chronologically rather than randomly (see Section 4.2), and for this reason the fraud rate is not identical across the partitions. Table 3.1 summarises the class balance observed across partitions.

The training partition used for the autoencoder is, by design, restricted to non-fraudulent transactions (see Section 4.5). Fraudulent transactions originally present in the first 60% of the data are folded into the classifier-fitting stage with the first half of the validation partition. This ensures that the XGBoost model sees every labelled fraud example available prior to the test period.

Partition	Rows	Fraud count	Fraud rate
Full dataset	590,540	20,663	3.499%
Training (autoencoder; fraud excluded)	342,336	0	.000%
Validation (full, 30%)	177,162	6,462	3.648%
Validation (clf-fit, 15%)	88,581	3,575	4.036%
Validation (val-calib, 15%)	88,581	2,887	3.259%
Test (10%)	59,054	2,213	3.747%

Table 3.1: Class balance across dataset partitions.

3.5 Implications for Modelling

Three properties of this dataset motivate specific design choices that are described in full in Chapter 4. Firstly, the width, redundancy, and structured missingness of the V block motivate the methodology of the dimensionality reduction steps. Secondly, the presence of partial account identifying fields, but no actual account identifier motivates the construction of the synthetic per-account UID, described in Section 4.3, which the sliding window and causal aggregate feature construction depends upon. Third, the low absolute number of fraud examples in any single partition and the high-dimension feature space, motivates the recursive feature elimination step described in Section 4.7 and the bootstrap-based threshold stabilisation procedure described in 4.9. These methods ensures that the train and validation splits produce a stable operating threshold.

Chapter 4: Methodology

This chapter describes, in the order in which they are applied, every stage of the experimental pipeline. This includes: feature engineering including dimensionality reduction, missingness handling, encoding, and causal behavioural aggregation; construction of per-account sliding-window sequences; the temporal convolutional autoencoder architechture; the feature fusion and RFECV-based feature selection step; the XGBoost classifier and its custom focal-loss training objective; the account-level fraud cascade rule; and the probability calibration and threshold-selection procedure. The implementation of this methodology was used to produce the results reported in Chapter 6.

4.1 System Overview

At the highest level, the experimental pipeline operates over five stages. Firstly, raw transaction and identity data are loaded, joined, sorted by time, and split into training, validation, and test partitions using fixed, non-overlapping time windows. In Stage 2, the feature engineering pipeline is fitted on the training partition, which avoids data leakage, and then applied identically to all three partitions, producing a wide, fully numeric feature matrix. In Stage 3, the feature matrix is organised into fixed-length, per-account sliding windows and a temporal convolutional autoencoder is trained, using only non-fraudulent windows, to reconstruct the sequence. After reconstruction, the fitted autoencoder is used to extract a fused representation - latent code, terminal-step reconstruction error, and window-level error statistics - for every window in every partition. In Stage 4, a recursive feature elimination with cross-validation selects a subset of this representation, and an XGBoost classifier is trained on the selected subset using a focal-loss objective and balanced instance weighting. Early stopping is used in this step, which is applied on a held-out validation partition. In Stage 5, XGBoost's raw fraud probability outputs are calibrated using isotonic regression and a decision threshold is selected to optimise an F-beta score

that favours recall. The threshold score is stabilised using bootstrap resampling, as a protection measure against the extreme class imbalance. As a result of a significant dataset characteristic - all transactions following a marked fraudulent transaction are characterised as fraud by the dataset, regardless of whether they are fraud or not - a causal account-level fraud cascade rule is applied during evaluation, before final metrics are computed on the held out test partition.

4.2 Data Loading and Time-Based Splitting

The transaction and identity tables are loaded and left-joined on `TransactionID`, and the resulting dataframe is sorted by `TransactionDT(Time)`[1]. The sorted dataframe is then split into three contiguous, non-overlapping blocks using a train/val/test split of 0.6/0.3/0.1; because the split points are computed as row count fractions of a time-sorted dataframe, every transaction in the validation partition occurred strictly after every transaction in the training partition, and every transaction in the test partition occurred strictly after every transaction in the validation partition. This methodological safeguard ensures that no stage of feature engineering, representation learning, feature selection, classifier training, calibration, or threshold selection can use information from the future relative to the data it is being evaluated on - which is a precondition for the reported test metrics being interpretable as an estimate of real-time performance.

The validation partition is further split in two equal halves: the first half (`clf_fit_df`) is concatenated with the full training partition (including fraud examples) and is used to fit the XGBoost classifier; the second half (`val_calib_df`) is reserved for early stopping, probability calibration, and threshold selection. This split ensures that the threshold selection and calibration stages are not evaluated on data that the classifier has optimised against.

4.3 Feature Engineering Pipeline

All feature engineering transformations are fitted exclusively on the training partition and then applied to the training, validation, and test partitions without refitting. This methodology prevents excessive leakage on transformations with learned parameters, and follows these steps, applied in order.

[1]Throughout the pipeline, a variety of columns have been renamed for ease of use or compliance with PEP-8 conventions. Their first reference will be written like so: `OriginalName(NewName)`, from which point they will be referred to as the name used in the pipeline.

4.3.1 V-column reduction

The 339 raw V columns are grouped by their exact per-row missingness pattern, by count of missing values each column has in the training partition. Columns sharing the same missing-value count are treated as belonging to the same underlying "NaN-count group", on the assumption that columns missing together are computed from the same upstream source and therefore highly related. Within each group, columns with zero variance are dropped outright, and remaining columns are reduced to a single representation head column using greedy clustering by pairwise absolute Pearson correlation. Starting from an unvisited column, all other unvisited column in the group with a correlation above 0.90 is added to a cluster and marked visited - only the first column of each cluster is then kept, with the rest dropped as redundant. After this methodology is applied to the dataframe, the 339 raw V columns are reduced to 228 kept columns: one column is dropped for zero variance and 110 columns are dropped as redundant across 14 distinct missingness groups.

4.3.2 Time and account features

An hour-of-day and day-of-week feature are derived from the relative timestamp as described in Section 3.2. A synthetic per-account identifier (`account_uid`) is constructed from three partial signals - `card1`, `addr1`,and `D1n`. `D1n` encodes a time-delta since account opening - calculated from subtracting the transaction's absolute day-index from `D1` to yield a constant variable across all transactions belonging to the same account. For rows where both `addr1` and `D1n` are present, the account identifier is the concatenation of `card1`, `addr1`, and `D1n`; for rows where either variable is missing, a unique fallback identifier is assigned per transaction. This prevents transactions with insufficient information to be grouped from being spuriously merged into a false shared account, at the cost of all transactions in this group being treated as individual accounts. This account identifier underpins both the sliding-window sequence construction in Section 4.4 and the causal per-account aggregate features described below, and is later used to define the fraud cascade rule in Section 4.8.

4.3.3 Derived features

The decimal component of the transaction amount is extracted and scaled to an integer, an indicator for whether the purchaser and recipient email domains match is computed, purchaser and recipient email domain suffixes are extracted, and, where

present, the leading token of the device operating system, browser, and device-info identity fields are extracted as coarser categorical features than the raw full strings.

4.3.4 Missingness indicators

For every column not protected from this step (identifiers, the label, time, and the raw fields used to build the account UID), if more than 30% of the columns values are missing in the training partition, a binary "is missing" indicator column is added. 181 indicator columns were computed for the 334 eligible columns in the dataframe.

4.3.5 Frequency encoding

. A fixed list of high-cardinality categorical fields (card identifiers, address codes, email domains, and several identity fields) are frequency-encoded: a lookup table mapping each observed category to its count in the training partition is fitted, and a new numeric column containing this count is appended for each field. The original column is not removed from the dataframe.

4.3.6 Categorical and numeric encoding

. Remaining object-typed columns are integer encoded using a sorted unique-value lookup, with unseen categories in validation or test mapped to a reserved "unknown" code. Remaining numeric columns are median-imputed, with the median computed on the training partition and reused for validation and test. The transaction amount is log1p-transformed and clipped at zero, which compresses its heavily right-skewed distribution.

4.3.7 Causal per-account aggregates

. After the transformations above, the three dataframe partitions are temporarily concatenated (in time order) so that account-level running statistics can be computed causally across partition boundaries without time leakage. For every account, and for every engineered numeric column and the transaction account, the pipeline computes: the number of prior transactions, a binary "is this account's first observed transaction" indicator, the time elapsed since the accounts previous transaction, the time elapsed since the accounts first observed transaction, and the running mean and running standard deviation of each feature prior to the current transaction. This is implemented as a cumulative-sum-based computation for the batch-case, and saved

as an explicit, incrementally-updatable UidAggregateStore object that maintains running counts, sums, and sums-of-squares per account and can featurise a new transaction and update its own state in a single call. This design is intended to support true streaming, transaction-by-transaction scoring in a deployed setting, and is discussed further in Section 4.10. Because these aggregates are computed strictly from prior transactions, an account's first-ever transaction has no valid running statistics; in that case, a fallback-value is substituted - the corresponding training-partition median for the feature, and a fixed value for "time since previous transaction".

After the application of the steps described above, the final feature matrix used for representation learning contains 1,519 columns, drawn from the reduced V block, engineered time and account-derived features, missingness indicators, frequency-encoded fields, encoded categorical and numeric fields, the log-transformed amount, and the full set of causal per-account aggregates.

4.4 Sliding-Window Sequence Construction

To expose sequential, per-account structure to the TCN-AE, the flat feature matrix is reorganised into fixed-length windows using a custom `SlidingWindowFraudDataset` Class. Rows are first grouped by `account_uid` and, within each account, ordered by their original (time-sorted) position. For each transaction at position end within its account's sequence, a window of length 16 (`seq_len` = `16`) is formed consiting of that transaction and up to the 15 immediately preceding transaction on the same account. If fewer than 16 prior transactions exist, the window is left-padded with zero vectors up to the required length, and an explicit binary mask channel - one value per time step, equal to 1 for real, observed transactions and 0 for padding - is concatenated onto the feature channels, so that the network can distinguish genuine zero-valued features from padding at every downstream computation. Features are standardised using a StandardScaler fitted only on the training partition's real unpadded rows, and the standardised values are clipped to the range [-10, 10] to bound the influence of residual outliers after standardisation. A window is labelled as fraudulent if any transaction within it is labelled fraudulent, so that the label attached to a window reflects the fraud status of the most recent transaction it represents.

Every window in the dataset is therefore associated with the identity of the account it belongs to; the dataset class exposes this account-id array so that both the fraud cascade rule (Section 4.8) and the blocking of leakage across accounts during evaluation can be implemented downstream.

4.5 Temporal Convolutional Autoencoder Architecture

4.5.1 Motivation for architectural choices

. The autoencoder is build from stacked temporal convolutional blocks rather than a recurrent architecture for the reasons discussed in Section 2.4: dilated causal convolutions provide a controllable, exponentially growing receptive field, are fully parallelisable across the sequence dimension during training, and, by construction, guarantee that the representation computed at the final time step of a window depends only on the time step and the preceding steps within the same window, never on any later transaction. This property is essential to a model intended for real-time, causal deployment.

4.5.2 Temporal block

. Each temporal block consists of two 1-D convolutional layers, each followed by a "chomp" operation that removes the trailing padding introduced by causal convolution, group normalisation, a ReLU non-linearity, and dropout (rate 0.2). Both convolutional layers use weight normalisation, a reparameterisation of the convolution's weight matrix into a magnitude and direction component to stabilise and accelerate training of deep convolutional stack. A residual connection adds the block's input to its output, passed through a 1x1 convolution to equalise these channel counts in the case that they differ, follwed by a final ReLU. This residual structure, following the original TCN formulation, allows for a deep structure with multiple dilations, without the optimisation difficulties that would otherwise accompany a deep, purely feed-forward convolutional stack.

4.5.3 Stack and dilation schedule

. Temporal blocks are stacked with a dilation rate that doubles at each successive block (1, 2, 4, ...). This is a standard TCN design that achieves an effective receptive field that grows exponentially with depth while keeping the number of parameters growing only linearly. The encoder uses three stacked blocks with channel widths of 32, 32, and 64 respectively ($\texttt{enc_channels} = (\texttt{32}, \texttt{32}, \texttt{64})$), and a kernel size of 3.

4.5.4 Padding-aware masking

. Because windows near the start of an account's history are left-padded, the model learns an explicit, trainable padding embedding vector to ensure it doesn't treat the zero-valued padding positions as genuine "all-zero" transactions. The trainable padding embedding vector is a one value per input feature channel that replaced the feature values at masked positions before the sequence enters the encoder, using the mask channel described in Section 4.4 to select, position by position, between real feature values and the learned padding embedding. The binary mask channel is retained as an additional input channel to the encoder, so the network as direct access to the pattern of which positions are real versus padded.

4.5.5 Encoder and latent bottleneck

. The masked, embedded sequence is passed through the three-block TCN encoder stack, and the representation at the final time-step of the sliding window is taken as the sequence summary. The sequence summary is linearly projected to a 32-dimensional latent vector using a fully connected layer. Using only the final time step, rather than pooling over the whole sequence or any other method, prevents any time leakage - the final-step hidden state is guaranteed to be a function only of the current and preceding transactions.

4.5.6 Decoder and reconstruction

. The latent vector is linearly projected to the first decoder channel width and repeated across all 16 time steps of the window, then passed through a mirrored TCN stack with reversed channel widths (relative to the encoder) and a final 1x1 convolution. This convolution maps back down to the original, unmasked feature dimensionality, and produces a full reconstructed window, `x_hat`, of the same shape as the unpadded input channels.

4.5.7 Reconstruction loss and masking

. The autoencoder is trained with a masked mean-squared-error objective: the squared difference between the reconstruction and the true input is computed at every position and every feature, multiplied by the binary mask - so that padded positions contribute nothing to loss - and averaged over the total number of valid position-feature entries in the batch. Training uses the Adam optimiser (learning

rate 1×10^{-4}, weight decay 1×10^{-5}), gradient-norm clipping at 1.0, and a reduce learning rate on plateau scheduler that halves the learning rate if the epoch-level reconstruction loss fails to improve for two consecutive epochs. The autoencoder is trained for a maximum of 12 epochs (`epochs_ae = 12`) on the training partition only, and the training partition supplied to the autoencoder is filtered down to non-fraudulent transactions only. This ensures the autoencoder learns a model of normal account behaviour exclusively - it has never been shown a confirmed fraudulent window during this stage.

4.6 Feature Fusion: From Reconstruction to a Classifier-Ready Representation

Rather than reducing each window to a single reconstruction-error scalar, the fitted autoencoder is used to compute a richer, fixed-length feature vector for every window, which is the raw input for the downstream classifier. This fused representation is computed by `classifier_input_features` and concatenates four components: (i) the 32-dimensional latent embedding, z, giving the classifier direct access to the network's learned summary of the account's recent behaviour; (ii) the terminal-step reconstruction error, computed as the absolute difference between the true and reconstructed feature values at the final time step only; (iii) three window-level reconstruction-error summary statistics - the mask-weighted mean absolute error across the window, the mask-weighted maximum absolute error across the window, and the mask-weighted standard deviation of the error across the window - which capture whether the entire recent sequence for this account was reconstructed poorly, rather than just the most recent step. The fused representation has 1,554 dimensions in total.

4.7 Feature Selection: RFECV

A 1,554-dimensional input is computationally expensive and statistically risky for a classifier trained on a relatively small amount of positive examples, and as such the pipeline applies recursive feature elimination with cross-validation (RFECV) to prune this representation before classifier training. RFECV is fitted using a random forest base estimator (150 trees, maximum depth 6, class-balanced sample weighting) as the scoring model, 5-fold cross-validation, average-precision as the scoring metric, and 10% of remaining features removed at each iteration. Given the size of the classifier-fitting partition, to ensure this task is computationally tractable, RFECV

is run on a class-balance-preserving subsample that is capped at 20,000 windows - all fraud windows are retained and the majority class is subsampled down to fill the remainder of the sample. 159 features were selected from the 1,554 fused features. The resulting boolean feature masked is stored on the autoencoder model object (`set_feature_map`) and applied consistently at all subsequent stages.

4.8 XGBoost Classifier

The selected 159-dimensional feature vectors, together with the fraud label, are used to train an XGBoost classifier using a histogram-based tree method, maximum tree depth 6, learning rate 0.05, 80% row and column subsampling per tree, and a minimum child weight of 2, for up to 500 boosting rounds with early stopping after 30 rounds without improvement, monitored on the calibration-half of the validation partition.

4.8.1 Custom focal-loss objective

. Rather than XGBoost's built-in logistic loss, the classifier is trained with a custom binary focal-loss objective, implemented directly as a Python function returning the per-example gradient and Hessian of the loss with respect to the model's raw score. For a true label y, predicted probability p (the result of the sigmoid function applied to the raw score), and the focal-loss parameters alpha (class-balancing weight, set to 0.55) and gamma (focusing parameter, set to 2.0), the per-example focal weight is computed as:

$$alpha \text{ or } 1 - alpha(\text{depending on true class}) = (1 - p_t)^{gamma}$$

where p_t is the predicted probability of the true class. This down-weights the gradient contribution of examples the model already classifies confidently and correctly, concentrating training signal on hard or misclassified examples as discussed in Section 2.2. In addition `fn_cost` (set to 1.2) is applied to positive fraud examples, over and above the alpha/gamma focal weighting, reflecting an explicit preference for penalising missed fraud more heavily than the focal-loss formulation would. Finally, the resulting focal weight is multiplied by a per-instance sample weight computed from inverse class frequency, so that class imbalance is addressed through two complementary mechanisms simultaneously - instance re-weighting and focal down-weighting of easy examples - rather than relying on either mechanism alone.

4.8.2 Early stopping metric

. Training is monitored, at each boosting round, using both the standard PR-AUC evaluation metric and a custom F-beta metric (beta = 1.5, evaluated at a fixed 0.5 probability threshold on the sigmoid-transformed raw score), which biases early stopping and model selection slightly toward recall relative to a beta = 1 (F1) criterion, consistent with a higher cost associated with missed fraud relative to false alarms. In the results reported in Chapter 6, the full 500 configured boosting rounds were run without early stopping being triggered, with the best iteration identified at round 498 (best validation PR-AUC of 0.3438).

4.9 Account-Level Fraud Cascade

. The IEEE-CIS dataset employs an operational assumption from the banking environment it is pulled from - any transaction in the 120 days following a fraudulent transaction is automatically flagged as fraudulent, whether or not it is made by the customer. This is implemented by the model using an `apply_fraud_cascade` function: for a given set of predictions and their associated account identifiers, it computes, per account and in time order, whether any prior transaction on that account has already been predicted fraudulent, and if so, marks the current transaction as "cascaded" and overrides its prediction to fraudulent regardless of the classifier's own score for that specific transaction.

This mechanism is used in two distinct ways. First, during evaluation, rows whose predict is a cascade override are excluded from the calibration and threshold-selection stages and from the reported ROC-AUC/PR-AUC ranking metrics, as this would inflate those metrics away from the quality of the model. Second, the final reported classification metrics include the cascade override, since this reflects the prediction an operational system would produce. In the validation run reported in Chapter 6, 495 of 88,581 calibration-partition rows were excluded from calibration as cascade-overriden.

4.10 Probability Calibration and Threshold Selection

The model addresses probability calibration issues in three stages.

4.10.1 Isotonic calibration

Using the calibration half of the validation partition, restricted to non-cascade-overidden rows, an isotonic regression calibrator is fitted mapping the classifier's raw probability outputs to empirically calibrated probabilities. Isotonic regression is chosen over a parametric alternative because it makes no assumption about the functional form of the miscalibration and can represent an arbitrary monotonic correction.

4.10.2 F-beta-optimal threshold

Using the calibrated scores, a decision threshold is chosen by sweeping the full precision-recall curve on the calibration partition and selecting the threshold that maximises an F-beta score with beta = 2.0. This weights recall twice as heavily as precision in the harmonic mean, reflecting the higher assumed cost of a missed fraud relative to a false alarm. In the run reported in Chapter 6, this single-split procedure selected a threshold of 0.3557.

4.10.3 Bootstrap threshold stabilisation

Because the calibration partition contains a limited number of positive examples, a threshold chosen from a single split is subject to non-trivial sampling variance. To reduce this variance, the pipeline performs 500 boostrap resamples of the non-cascaded calibration partition, recomputes the F-beta-optimal threshold on each resample, discards any resample with fewer than two positive examples or with no negative examples, and takes the median of the resulting distribution of the thresholds as the final threshold. In the run reported in Chapter 6, this procedure produced a stabilised threshold of 0.4045.

4.11 Held-Out Test Evaluation and Uncertainty Quantification

The final, calibrated model is applied, exactly once, to the held-out test partition. Predictions are produced by extracting the fused feature representation for every test window, applying the stored feature mask, obtaining calibrated probabilities from the trained booster and isotonic calibrator, thresholding at the stabilised threshold, and applying the account-level fraud cascade rule. To quantify the sampling uncertainty of the headline F1 metric on this single held-out partition, a further 500-resample

bootstrap is performed directly on the final test predictions. This is reported in full in Chapter 6.

4.12 Inference Artefacts and Real-Time Deployment Considerations

Although this research evaluates the system in an offline, batch setting, the pipeline is designed with streaming deployment in mind, and contains a set of inference artefacts for that purpose described above, with the most important being a "warm" UidAggregateStore object. This object is initialised from the entire time range of data seen during training, validation, and testing, and has a a `featurize_and_update` method that can compute causal per-account aggregate features for incoming transactions in a single call. This design means that, in principle, a newly arriving transaction can be featurised and scored without re-scanning that account's full transaction history. A full production deployment would also require wrapping the trained TCN-AE and the XGBoost classifier in a low-latency serving layer and maintaining a rolling window of each account's most recent 16 transactions in a fast key-value store.

Chapter 5: Experimental Setup and Implementation

5.1 Software and Hardware

The pipeline is implemented in Python, using `PyTorch` for the TCN-AE, XGBoost's native training API for the classifier, and `scikit-learn` for the isotonic calibrator, the RFECV feature-selection procedure, the StandardScaler, and the standard classification metrics. `NumPy` and `pandas` are used throughout for data loading, feature engineering, and the sliding-window and causal-aggregate computations. The training has the capability to select a CUDA device if one is available, but otherwise falls back to CPU.

5.2 Reproducibility

A single global random seed (42) is used to initialise Python's random module, `NumPy`'s random number generator, and `PyTorch`'s CPU and CUDA random number generators, applied at the start of the run before any stochastic operation occurs. The same seed is threaded through to the RFECV cross-validation folds, the XGBoost training procedure, and both bootstrap procedures.

5.3 Hyperparameter Summary

Table 5.1 summarises the principal hyperparameters used in the run reported in this research.

Component	Hyperparameter	Value
Data split	Train / validation fraction	0.6 / 0.3
Data split	Classifier-fit / calibration split of validation	0.5 / 0.5
Feature engineering	Missingness indicator threshold	30% missing in training partition
Feature engineering	V-column correlation threshold	0.90
Sliding window	Sequence length	16 transactions
Sliding window	Stride	1
Sliding window	Feature clip range (post-standardisation)	[-10, 10]
TCN autoencoder	Encoder channel widths	32, 32, 64
TCN autoencoder	Kernel size	3
TCN autoencoder	Dilation schedule	1, 2, 4
TCN autoencoder	Latent dimension	32
TCN autoencoder	Dropout	0.2
TCN autoencoder	Optimiser	Adam, lr = 1e-4, weight decay = 1e-5
TCN autoencoder	LR schedule	ReduceLROnPlateau, factor 0.5, patience 2 epochs
TCN autoencoder	Max epochs	12
TCN autoencoder	Gradient clipping	Max norm 1.0
Feature selection	Method	RFECV, RandomForest base estimator
Feature selection	Base estimator	150 trees, max depth 6, class-balanced weights
Feature selection	CV folds	5
Feature selection	Scoring metric	Average precision (PR-AUC)
Feature selection	Subsample cap	20,000 windows (fraud fully retained)
XGBoost	Max tree depth	6
XGBoost	Learning rate	0.05
XGBoost	Subsample/colsample_bytree	0.8 / 0.8
XGBoost	Min child weight	2
XGBoost	Max boosting rounds	500
XGBoost	Early stopping	30 rounds, on validation PR-AUC / F1.5
XGBoost	Focal loss alpha / gamma	0.55 / 2.0
XGBoost	False-negative gradient multiplier	1.2
Threshold selection	F-beta for optimal threshold	2.0
Threshold selection	Bootstrap resamples	500
Test F1 confidence interval	Bootstrap resamples	500
Global	Random seed	42

Table 5.1: Hyperparameter summary

5.4 Autoencoder Checkpointing and Training Behaviour

The autoencoder training loop persists the model's to disk after every epoch that improves on the best reconstruction loss observed so far, and the top-level training script checks for an existing checkpoint before beginning autoencoder training: if a checkpoint is found, it is loaded directly and autoencoder training is skipped entirely, unless retraining is forced via a command-line flag. All downstream stages are still executed if autoencoder training is skipped. The loading step is wrapped to catch a mismatch between a saved checkpoint feature dimensionality and the current run's feature dimensionality.

The autoencoder wad trained from scratch for the full 12 configured epochs for the run reported in Chapters 6 and 7, producing the masked reconstruction-loss curve shown in Table 5.2.

The loss falls steeply over the first three epochs as the network learns the coarse structure of normal account behaviour, and then continues to improve at a steadily

Epoch	Reconstruction loss
1	0.676788
2	0.460734
3	0.405118
4	0.372442
5	0.351900
6	0.338510
7	0.331238
8	0.323879
9	0.316277
10	0.309869
11	0.304742
12	0.300488

Table 5.2: Table 5.2: Autoencoder masked reconstruction loss by epoch

descreasing rate through epoch 12. No epoch shows an increase in loss, which is consistent with the ReduceLROnPlateau schedule not intervening aggressively, and with 12 epochs being a reasonable, if not necessarily final, stopping point. The checkpoint saved at epoch 12 is used for all downstream feature extraction, RFECV, and classifier training.

5.5 Evaluation Protocol Summary

For clarity, the full evaluation protocol used to produce every metric reported is summarised here: (1) fit all feature-engineering transforms on the training partition; (2) train the autoencoder on non-fraudulent training-partition windows; (3) extract fused features for the classifier-fit and calibration partitions using the fitted autoencoder; (4) run RFECV on the classifier-fit partition to obtain a feature mask; (5) train XGBoost, with the focal-loss objective, on the classifier-fit partition's selected features, monitoring the calibration partition for early stopping; (6) fit the isotonic calibrator on the cascade-excluded calibration partition; (7) select an F-beta-optimal threshold and stabilise it via bootstrap resampling; (8) apply the fully-fitted pipeline to the held-out test partition; (9) quantify the sampling uncertainty of the headline test F1 via a further bootstrap directly on the test predictions.

Chapter 6: Results

This chapter reports the results of a full run of the pipeline described in Chapters 4 and 5.

6.1 Dataset and Feature Engineering Summary

Loading and sorting the joined transaction/identity tables by time produced 590,540 transactions in total, of which 20,663 (3.4990%) are confirmed fraud. The V-column reduction step reduced the 339 raw anonymised V columns to 228 kept columns as described in Section 4.3.1. Missingness indicators were added for 181 of the 334 eligible columns exceeding the 30%-missing threshold in the training partition. After all feature engineering steps, the final feature engineering steps, the final feature matrix used for representation learning contains 1,519 columns. Table 6.1 restates the partition-level class balance introduced in Section 3.4.

Partition	Rows	Fraud	Rate
Full dataset	590,540	20,663	3.4990%
AE training (fraud excluded)	342,336	0	0.0000%
Validation (full)	177,162	6,462	3.6475%
Classifier-fit half of validation	88,581	3,575	4.0359%
Calibration half of validation	88,581	2,887	3.2592%
Test (held-out)	59,054	2,213	3.7474%

Table 6.1: Table 6.1: Partition sizes and class balance

6.2 Autoencoder Training

The run reported here trained the autoencoder from scratch for the full 12 configured epochs, following the checkpointing behaviour described in Section 5.4. The masked reconstruction loss, logged at the end of every epoch, fell monotonically from 0.676788 at epoch 1 to 0.300488 at epoch 12 (Table 5.2), with the steepest improvement occurring in the first three epochs. The epoch-12 checkpoint was used for all

downstream feature extraction, feature selection, classifier training, calibration, and evaluation described in the remainder of this chapter.

6.3 Feature Selection

RFECV was applied to the 1,554-dimensional fused feature representation extracted from the classifier-fit partition, and selected 159 features using 5-fold cross-validated average precision. This selected subset was fixed and reused for every subsequent stage of the pipeline.

6.4 XGBoost Classifier Training

Table 6.2 reports a representative subset of the training curve, logged every 50 boosting rounds, showing the train/validation PR-AUC and F1.5 metrics used respectively for the standard XGBoost evaluation log and the custom early-stopping metric.

Round	Train aucpr	Train F1.5	Val aucpr	Val F1.5
0	0.8442	0.1058	0.2477	0.0987
50	0.9176	0.3082	0.3590	0.2576
100	0.9392	0.3554	0.3692	0.2790
150	0.9535	0.3965	0.3726	0.2915
200	0.9636	0.4297	0.3759	0.3024
250	0.9717	0.4655	0.3777	0.3116
300	0.9780	0.5004	0.3781	0.3177
350	0.9829	0.5309	0.3782	0.3249
400	0.9867	0.5625	0.3789	0.3316
450	0.9896	0.5930	0.3806	0.3372
498 (best)	-	-	0.3438 (best aucpr)	-
499 (stopped)	0.9917	0.6187	0.3809	0.3434

Table 6.2: Table 6.2: XGBoost training curve (selected rounds)

Training ran the full 500 configured rounds without triggering early stopping, with the best iteration identified at round 498. The training aucpr and F1.5 curves climb steadily and without plateauing, while the corresponding validation curves rise much more slowly and flatten well below the training curves. This widening train/validation gap is a visible sign of the classifier's capacity to fit the training data far more closely than it can generalise - likely due to the high dimensionality of the fused representation relative to the number of confirmed fraud examples available. This is one reason the

feature-selection stage and validation-monitored early-stopping is important to the final result, even if early-stopping is not triggered.

6.5 Calibration and Threshold Selection

Upon applying the account-level fraud cascade rule, as described in Section 4.9, to the dataframe, 495 of the caibration-partition rows were identified as cascade-overridden and excluded from the isotonic calibration fit and from the reported ranking metrics. On the remaining rows, after isotonic calibration, the classifier achieved:

- ROC-AUC: 0.8374
- PR-AUC: 0.3662
- F1: 0.3966
- F2: 0.3449
- Recall: 0.3173
- Precision: 0.5289
- False negatives: 1,971

As described in Section 4.10.2, the single split, F-beta = 2.0-optimal threshold on the calibration partition was 0.3557. Following the threshold-stailisation procedure, the stabilised threshold was 0.4045.

6.6 Held-Out Test Performance

The fully-fitted pipeline was applied once to the held-out test partition of 59,054 transactions, containing 2,213 confirmed fraudulent transactions. The headline results are:

Metric	Value
ROC-AUC	0.8594
PR-AUC	0.4254
Calibrated threshold	0.4045
F1 score	0.4240
F2 score	0.3623
Recall	0.3303
Precision	0.5919
False negatives	1,482

Table 6.3: Table 6.3: Held-out test set performance

The corresponding confusion matrix, with rows index by true label and columns by predicted label is:

	Predicted legitimate	Predicted fraud
Actual legitimate	56,337	504
Actual fraud	1,482	731

Table 6.4: Table 6.4: Test set confusion matrix

The full per-class classification report is reproduced in Table 6.5.

Class	Precision	Recall	F1	Support
0	0.9744	0.9911	0.9827	56,841
1	0.5919	0.3303	0.4240	2,213
Accuracy			0.9664	59,054
Macro average	0.7831	0.6607	0.7033	59,054
Weighted average	0.9600	0.9664	0.9617	59,054

Table 6.5: Table 6.5: Test set classification report

6.7 Uncertainty Quantification on the Test Metric

To quantify the sampling variability of the headline test F1 score arising from the finite size of the test partition, a further 500-resample bootstrap was performed directly on the final, cascade-adjusted test predictions. This produced a bootstrap mean test F1 of 0.4247, with a 95% confidence interval of [0.4051, 0.4450].

6.8 Summary of Findings

Taken together, the results in this chapter show a model that is a strong ranker of fraud risk (ROC-AUC 0.8594, PR-AUC 0.4254 with a 3.75% base rate), but a com-

paratively conservative classifier at the selected operating point. At the threshold selected, it recovers roughly one in three confirmed frauds, while maintaining a precision of 59.2%. This threshold is a free parameter of the deployed system rather than an intrinsic property of the trained model, meaning it is realistically possible to improve these rates. Chapter 7 discusses this precision-recall trade-off, and the operational choices it implies.

Chapter 7: Discussion

7.1 Interpreting the Ranking Performance

The test-set ROC-AUC of 0.8594 indicates that, across all possible thresholds, the model ranks a randomly chosen fraudulent transaction above a randomly chosen legitimate transaction roughly 86% of the time, which is a solid, though not exceptional, result relative to the general fraud-detection literature, where reported ROC-AUC figures on similarly imbalanced public benchmarks (including prior published work on this same IEEE-CIS dataset) commonly fall in the 0.85–0.95 range depending on the amount of feature engineering, ensembling, and external data enrichment applied. The PR-AUC of 0.4254 is the more informative of the two ranking metrics given the class imbalance involved, and is more than eleven times higher than the 0.0375 PR-AUC a random or "no-skill" classifier would achieve at this test-set base rate, which is a clearer and more conservative signal that the fused TCN-autoencoder-plus-XGBoost representation is genuinely discriminative of fraud, well beyond what a naive baseline would provide, even though it does not translate into a proportionally large recall at any single operating threshold, for reasons discussed in Section 7.2.

7.2 The Precision-Recall Trade-off at the Selected Operating Point

At the bootstrap-stabilised threshold of 0.4045, the model recovers 33.0% of confirmed fraud at a precision of 59.2%. This is a materially conservative operating point: it is chosen by an F-beta = 2.0 criterion, which does favour recall over precision within the space of thresholds evaluated on the calibration partition, and yet the resulting threshold still yields a comparatively low realised recall in absolute terms. Two factors, both traceable directly to the methodology in Chapter 4, help explain this outcome and are worth making explicit rather than treating the recall figure as simply "the model's ceiling." First, the bootstrap stabilisation step moved the threshold from

0.3557 (the single-split F-beta-optimal value) to 0.4045, a non-trivial upward (more conservative) shift. This shift is a direct consequence of designing the stabilisation procedure to take the median threshold across 500 bootstrap resamples of a calibration partition containing under three thousand positive examples: individual resamples in which the classifier's precision-recall curve is comparatively "flat" near the recall-favouring end will tend to select noticeably lower thresholds on some draws, but the procedure as implemented is symmetric and takes the central tendency of the resulting distribution rather than, say, its lower quantile, so it does not explicitly bias toward the more aggressive (higher-recall) end of the sampling distribution. An institution with a stated preference for erring toward catching more fraud at the cost of additional false alarms could straightforwardly modify this stabilisation rule — for instance, taking a lower percentile of the bootstrap threshold distribution rather than the median, or increasing the F-beta parameter beyond 2.0 — and this is one of the more immediately actionable levers identified by this analysis (see Section 8.3). Second, and more fundamentally, the achievable recall at any fixed precision level is bounded by how separable the fused feature representation actually is for this population of frauds, and the results in Table 6.2 show a training PR-AUC approaching 0.99 alongside a validation PR-AUC plateauing around 0.38, a gap that is characteristic of a model with the statistical capacity to memorise idiosyncratic patterns in the 6,462 fraud examples seen during classifier fitting, without those patterns generalising equally well to the 2,213, necessarily different, fraud examples in the held-out test period. This generalisation gap — rather than any single hyperparameter choice — is the primary factor limiting how much further recall could be extracted from this architecture without either materially more fraud-labelled training data, materially richer input features (Section 7.5), or a materially more constrained model.

7.3 The Effect of the Fraud Cascade Rule

The account-level fraud cascade rule (Section 4.9) affected 495 of 88,581 calibration-partition rows in the run reported here — a small fraction (0.56%) of the calibration partition overall, though a substantially larger fraction of the fraud-positive rows specifically, since cascaded predictions are, by construction, concentrated on accounts already flagged as fraudulent. Excluding these rows from the ranking metrics (ROC-AUC, PR-AUC) and from the calibrator's training data is a deliberate methodological choice: including cascade-forced predictions in these ranking metrics would credit the model with "detecting" transactions that were, in fact, flagged purely because an

earlier transaction on the same account had already crossed the threshold, which conflates the discriminative quality of the model's own per-transaction scores with the coverage effect of a simple propagation rule. That said, the classification metrics reported in Chapter 6 (F1, F2, precision, recall, the confusion matrix) do include cascade-forced predictions, since these reflect what an operational system, applying this policy, would actually report to a downstream reviewer, and this is why the reported test recall (33.0%) should be read as inclusive of, not in addition to, whatever incremental coverage the cascade rule contributes.

7.4 Relationship to the Research Questions

Returning to the four research questions posed in Section 1.3: the results support an affirmative, if qualified, answer to Research Question 1 — the fused TCN-autoencoder representation, and specifically the combination of its latent embedding with reconstruction-error statistics, was demonstrably useful to the downstream classifier, since RFECV retained 159 of the 1,554 fused dimensions as informative under cross-validated average precision, rather than collapsing toward the small subset of raw tabular features alone, though this thesis's ablation coverage (Section 7.6) is not extensive enough to attribute a precise share of the final PR-AUC to the autoencoder-derived features specifically as opposed to the retained raw tabular features. On Research Question 2, the achieved operating point (59.2% precision at 33.0% recall, PR-AUC 0.4254) establishes a concrete, reproducible point on this dataset's precision-recall frontier for this architecture, though it should be read as one point on a frontier that a deploying institution could shift along by adjusting the threshold-selection procedure, not as evidence of an unconditional recall ceiling. On Research Question 3, the fraud cascade analysis in Section 7.3 shows that accounting for within-account correlation materially changes which rows are eligible to influence the calibration and ranking-metric stages of the pipeline, even though its numerical impact on the specific run reported here was modest, and establishes a template for separating "genuine per-transaction discrimination" from "propagation-rule coverage" in the reported metrics. On Research Question 4, the shift from a single-split threshold of 0.3557 to a bootstrap-stabilised threshold of 0.4045 is direct, quantitative evidence that a naive, single-split threshold-selection procedure would have been meaningfully unstable on this dataset's limited pool of fraud examples, and that the bootstrap stabilisation step is not a redundant formality.

7.5 Limitations

Several limitations of the present study should be acknowledged directly, rather than left implicit in the results of Chapter 6. First, the achieved recall (33.0%) is, in absolute terms, modest for a production fraud detection system, and an institution deploying this specific configuration would need to accept that roughly two-thirds of confirmed fraud, at this specific threshold, would go undetected by the model alone (though in practice such a model is rarely the sole line of defence, and typically operates alongside rule-based systems, manual review queues, and network-level signals not modelled here). Second, the anonymised nature of the IEEE-CIS V, C, D, and id_ feature blocks means that neither the feature-engineering choices in Chapter 4 nor the resulting model can be fully interpreted in domain terms (it is not possible to state, for example, that a particular retained feature corresponds to "distance from the cardholder's home address," even where such an interpretation might be plausible), which limits the direct transferability of specific feature-engineering findings to a differently structured production dataset, even though the methodology (missingness-aware dimensionality reduction, causal account aggregation, sliding-window sequence construction) transfers readily. Third, the synthetic account identifier constructed in Section 4.3, built from card1, addr1, and a normalised D1 field, is itself an approximation — it is not a guaranteed one-to-one mapping to a true underlying account or cardholder, and errors in this construction (either falsely merging distinct accounts or falsely splitting a single account into multiple synthetic identifiers) would propagate into every downstream account-level feature and into the fraud cascade rule itself; this thesis does not have access to ground-truth account identifiers against which to validate this construction directly. Fourth, the evaluation in this thesis is entirely offline and batch-based; while the time-based split is designed to simulate a forward-looking deployment, and the persisted UidAggregateStore artefact is designed to support true streaming inference, no experiment in this thesis actually measures inference latency, throughput, or performance under a genuinely streaming, transaction-by-transaction replay, so claims about "real-time" readiness in the thesis title should be understood as referring to the causal, leakage-free design of the pipeline rather than to a measured latency benchmark. Fifth, the reported figures are drawn from a single trained instance of the pipeline (one autoencoder checkpoint, one XGBoost training run, one RFECV feature mask), and while the seed-based reproducibility described in Section 5.2 and the bootstrap-based uncertainty quantification in Section 6.7 both increase confidence that this single run is representative, a fully rigorous treatment would

repeat the entire pipeline — including autoencoder training and RFECV — across multiple random seeds and report variance across runs, which this thesis, for reasons of computational cost, does not do.

Chapter 8: Conclusion and Future Work

8.1 Summary of the Thesis

This thesis set out to design and evaluate a hybrid temporal convolutional autoencoder and XGBoost architecture for real-time-oriented credit card fraud detection, addressing the extreme class imbalance, temporal correlation, and account-level structure that characterise the problem. A leakage-conscious feature engineering pipeline reduced a wide, highly redundant, and structurally missing anonymised feature space (339 raw V columns, and a further large block of identity and behavioural fields) down to 1,519 engineered features, while adding explicit missingness indicators and strictly causal, incrementally-updatable per-account behavioural aggregates. A temporal convolutional autoencoder, trained exclusively on non-fraudulent, per-account sliding windows with a masked reconstruction objective, was used not as a standalone anomaly detector but as a learned feature extractor, whose latent embedding and reconstruction-error diagnostics were fused with the raw feature space, pruned from 1,554 to 159 dimensions via cross-validated recursive feature elimination, and passed to an XGBoost classifier trained under a custom focal-loss objective with an explicit false-negative penalty. Isotonic calibration and a bootstrap-stabilised, F-beta-optimal decision threshold were used to convert the classifier's raw outputs into an operational decision, and an account-level fraud cascade rule was applied both to correct the reported ranking metrics for within-account correlation and to reflect a realistic operational escalation policy. On a strictly time-forward, held-out test partition of 59,054 transactions containing 2,213 confirmed frauds, the resulting system achieved a ROC-AUC of 0.8594, a PR-AUC of 0.4254, a precision of 0.5919, and a recall of 0.3303 (F1 = 0.4240, bootstrap 95% CI [0.4051, 0.4450]).

8.2 Answering the Research Questions

The evidence gathered in this thesis supports the conclusion that a temporal convolutional autoencoder, trained purely on legitimate account behaviour and used as a feature-extraction front end rather than a standalone anomaly scorer, contributes genuinely useful signal to a downstream gradient-boosted classifier for this task, that the resulting hybrid architecture achieves a materially above-baseline, though not exceptional, position on the precision-recall frontier for this dataset, that accounting for within-account correlation via an explicit fraud cascade rule is methodologically important both for the honesty of reported ranking metrics and as a model of realistic operational policy, and that naive, single-split threshold selection is measurably unstable on a dataset with this few positive examples in its calibration partition, making the bootstrap-based stabilisation procedure a worthwhile addition rather than an unnecessary complication.

8.3 Directions for Future Work

Building directly on the limitations identified in Section 7.5 and Section 7.6, four directions for future work stand out as both high-value and practically achievable extensions of the present pipeline.

8.3.1 Ablation of the autoencoder contribution

As identified in Section 7.6, training and evaluating an otherwise identical pipeline with the autoencoder-derived features (the 32-dimensional latent embedding and the 3 window-level error statistics) removed prior to RFECV, alongside a variant trained on raw tabular features alone with no sequence model at all, would directly quantify how much of the reported PR-AUC is attributable to the temporal convolutional autoencoder specifically, and is the most immediately valuable next experiment given the infrastructure already in place.

8.3.2 Cost-sensitive, institution-specific threshold selection

The threshold-selection procedure in Section 4.10 optimises a generic F-beta = 2.0 criterion, which is a reasonable default but does not incorporate the actual, typically highly skewed, distribution of transaction amounts (a missed $2,000 fraud and a missed $20 fraud are not equally costly, and neither is a false alarm on each) nor an institution's actual per-transaction review capacity. Replacing the F-beta

threshold criterion with a directly cost-weighted objective — for example, selecting the threshold that minimises expected monetary loss, using transaction amount as a per-example cost weight for false negatives and a fixed operational cost per false positive for manual review — would produce a threshold, and quite possibly a different threshold-stabilisation procedure, more directly aligned with deployment incentives than the current F-beta criterion.

8.3.3 Graph-based and network-level features

The current pipeline's account-level aggregates are strictly per-account and temporal; they do not model relationships between accounts (for example, multiple accounts transacting with the same merchant, sharing a device fingerprint, or sharing partial address or card information beyond the specific fields used to construct the synthetic account UID in Section 4.3). Constructing a bipartite or multipartite graph connecting accounts, devices, and merchants, and extracting graph-based features (such as account-device-merchant connectivity statistics or graph neural network embeddings) as an additional input to the fused representation, is a natural extension well supported by the existing fraud-detection literature and could plausibly recover some of the recall this thesis's purely per-account, purely temporal representation is unable to capture.

8.3.4 True streaming evaluation

The UidAggregateStore artefact described in Section 4.12 was designed specifically to support incremental, transaction-by-transaction featurisation without re-scanning an account's full history, but this thesis evaluates the pipeline only in a batch, offline setting. Building a genuine streaming replay harness — feeding the held-out test partition through the persisted inference artefacts one transaction at a time, in strict time order, measuring per-transaction latency and throughput, and confirming numerically that the streaming featurisation path reproduces the batch featurisation path exactly — would close the gap between the causal, leakage-free design of the pipeline documented in this thesis and a genuinely demonstrated real-time deployment, and is a natural, comparatively low-risk final step toward the "real-time" framing in the thesis title.

8.3.5 Repeated-seed variance estimation

Finally, repeating the full pipeline — including autoencoder training and RFECV feature selection, both of which are stochastic and were not varied across multiple seeds in the present study — across a small number of additional random seeds, and reporting the resulting variance in the headline test metrics alongside the single-run bootstrap confidence interval already reported in Section 6.7, would strengthen the evidential basis for the specific point estimates reported in this thesis and distinguish genuine architectural effects from run-to-run noise in future comparative work.

8.4 Closing Remarks

Credit card fraud detection remains a problem in which no single architectural choice is likely to deliver a definitive solution, both because the underlying phenomenon is adversarial and non-stationary and because the practical cost of a false negative and a false positive are institution-specific rather than universal constants that any offline benchmark can fully capture. This thesis has aimed to contribute a carefully specified, leakage-conscious, and honestly evaluated hybrid architecture — combining self-supervised sequence representation learning with a cost-sensitive, gradient-boosted classifier — together with an evaluation methodology (time-based splitting, account-level cascade correction, bootstrap-based threshold stabilisation and uncertainty quantification) that is, in the view of this thesis, at least as important a contribution as the specific numerical results obtained. The gap between the strong ranking performance (PR-AUC 0.4254, more than eleven times the no-skill baseline) and the more modest realised recall (33.0%) at the chosen operating threshold is not, this thesis argues, a failure of the underlying representation so much as a reminder that the choice of operating threshold is itself a first-class design decision, one that should be made deliberately, in direct reference to an institution's actual cost structure, rather than inherited by default from a generic statistical criterion.

Bibliography

[1] Andrew. Parry. *Card-not-present (CNP) transactions and fees*. 2025. URL: https://www.merchantsavvy.co.uk/card-processing-fees/card-not-present-cnp-transactions/.

[2] S. Chen, F. Ding, D. Buil-Gil, et al. "The impact of Covid-19 lockdown on fraud in the UK." In: *Humanities and Social Sciences Communications* 11.1676 (2024). DOI: https://doi.org/10.1057/s41599-024-04201-z.

[3] Diego Carrillo. *Understanding APP fraud regulation in the UK*. 2025. URL: https://www.hsbcinnovationbanking.com/sg/en/resources/understanding-app-fraud-regulation-in-the-uk.

[4] G. Charizanos, H. Demirhan, and D. Icen. "An online fuzzy fraud detection framework for credit card transactions". In: *Expert Systems with Applications* (2024). DOI: 10.1016/j.eswa.2024.124127.

[5] Andrea Dal Pozzolo, Olivier Caelen, Yann-Ael Le Borgne, et al. *Learned lessons in credit card fraud detection from a practitioner perspective - sciencedirect*. Aug. 2014. URL: https://www.sciencedirect.com/science/article/abs/pii/S095741741400089X.

[6] Haibo He and E.A. Garcia. "Learning from Imbalanced Data". In: *IEEE Transactions on Knowledge and Data Engineering* 21.9 (Sept. 2009), pp. 1263–1284. DOI: 10.1109/tkde.2008.239.

[7] Tsung-Yi Lin, Priya Goyal, Ross Girshick, et al. *Focal loss for dense object detection*. Feb. 2018. URL: https://arxiv.org/abs/1708.02002.

[8] Mayu Sakurada and Takehisa Yairi. *Anomaly detection using autoencoders with nonlinear dimensionality reduction*. Dec. 2014. URL: https://oamonitor.ireland.openaire.eu/rpo/rcsi/search/publication?pid=10.1145%2F2689746.2689747.

[9] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. *An empirical evaluation of generic convolutional and recurrent networks for Sequence Modeling*. Apr. 2018. URL: https://arxiv.org/abs/1803.01271.

[10] Kuldeep Randhawa, Chu Kiong Loo, Manjeevan Seera, et al. "Credit card fraud detection using ADABOOST and majority voting". In: *IEEE Access* 6 (2018), pp. 14277–14284. DOI: 10.1109/access.2018.2806420.

[11] Ugo Fiore, Alfredo De Santis, Francesca Perla, et al. "Using generative adversarial networks for improving classification effectiveness in credit card fraud detection". In: *Information Sciences* 479 (Apr. 2019), pp. 448–455. DOI: 10.1016/j.ins.2017.12.030.

[12] IY. Hafez, AY. Hafez, A. Saleh, et al. "A systematic review of AI enhanced techniques in credit card fraud detection." In: *Journal of Big Data* 12 (1 2025). DOI: https://doi.org/10.1186/s40537-024-01048-8.

[13] C. Wang, Q. Liu, L. Qu, et al. "Self-attention-enhanced Temporal Convolutional Networks for Anomaly Detection in Time Series." In: *Proceedings of the 2025 4th International Conference on Cryptography, Network Security and Communication Technology*. 2025. DOI: https://doi.org/10.1145/3723890.3723911.

[14] S. Gopali, F. Abri, S. Siami-Namini, et al. "A Comparison of TCN and LSTM Models in Detecting Anomalies in Time Series Data." In: *2021 IEEE International Conference on Big Data (Big Data)*. 2021. DOI: https://doi.org/10.1109/BigData52589.2021.9671488.

[15] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson. "PaySim: A financial mobile money simulator for fraud detection". In: *The 28th European Modeling and Simulation Symposium-EMSS*. 2016. URL: https://www.researchgate.net/publication/313138956_PAYSIM_A_FINANCIAL_MOBILE_MONEY_SIMULATOR_FOR_FRAUD_DETECTION.

[16] CIFAS Newsroom. *£9.4 billion stolen from UK consumers in a year*. 2025. URL: https://www.cifas.org.uk/newsroom/9.4billion_stolenfromconsumers.

[17] J. K. Afriyie, K. Tawiah, W. A. Pels, et al. "A supervised machine learning algorithm for detecting and predicting fraud in credit card transactions". In: *Decision Analytics Journal* 6.100163 (2023). DOI: 10.1016/j.dajour.2023.100163.

[18] C. Joshi, I. Kaloskampis, and L. Nolan. *Generative adversarial networks (GANs) for synthetic dataset generation with binary classes.* 2019. URL: https://datasciencecampus.ons.gov.uk/projects/generative-adversarial-networks-gans-for-synthetic-dataset-generation-with-binary-classes/.

[19] F. Gurcan and A. Soylu. "Synthetic Boosted Resampling Using Deep Generative Adversarial Networks: A Novel Approach to Improve Cancer Prediction from Imbalanced Datasets". In: *Cancers* 16 (23 2024). DOI: https://doi.org/10.3390/cancers16234046.

[20] Y. Yang, H. A. Korshidi, and U. Aickelin. "A review on over-sampling techniques in classification of multi-class imbalanced datasets: insights for medical problems." In: *Frontiers in Digital Health* 6.1430245 (2024). DOI: 10.3389/fdgth.2024.1430245.

[21] W. Wang and D. Sun. "The improved AdaBoost algorithms for imbalanced data classification." In: *Information Sciences* 563 (2021), pp. 358–374. DOI: 10.1016/j.ins.2021.03.042.

[22] M. Thill, W. Konen, H. Wang, et al. “Temporal convolutional autoencoder for unsupervised anomaly detection in time series”. In: *Applied Soft Computing* 112.107751 (2021). DOI: 10.1016/j.asoc.2021.107751.

[23] B. Wiese and C. Omlin. “Innovations in Neural Information Paradigms and Applications”. In: Studies in Computational Intelligence. Springer Berlin, Heidelberg, 2009. Chap. Credit Card Transaction, Fraud Detection, and Machine Learning: Modelling Time with LTSM Recurrent Neural Networks, pp. 231–268. DOI: 10.1007/978-3-642-04003-0_10.

[24] Y. Xie, G. Liu, R. Cao, et al. “A Feature Extraction Method for Credit Card Fraud Detection.” In: *2019 2nd International Conference on Intelligence Autonomous Systems (ICoIAS)*. 2019. DOI: 10.1109/ICoIAS.2019.00019.

[25] D. Varmedja, M. Karanovic, S. Sladojevic, et al. “Credit Card Fraud Detection - Machine Learning methods.” In: *2019 18th International Symposium INFOTEH-JAHORINA (INFOTEH)*. 2019. DOI: 10.1109/INFOTEH.2019.8717766.

[26] K. R. Ahmed, A. Rohan, S. A. Mitu, et al. “Improving Financial Security: A Hybrid AI-Based Credit Card Fraud Detection Framework.” In: *2025 5th International Conference on Intelligent Technologies (CONIT)*. 2025. DOI: 10.1109/CONIT65521.2025.11167292.

[27] M. I. Ayub, B. Bhattacharjee, P. Akter, et al. “Deep Learning for Real-Time Fraud Detection: Enhancing Credit Card Security in Banking Systems”. In: *The American Journal of Engineering and Technology* 7 (4 2025), pp. 141–150. DOI: 10.37547/tajet/Volume07Issue04-19.